



GPU Acceleration for Classic Machine-Learning

Tim Head

Own Your Datascience GOSIM 2026 | 5 May 2026



Classic Machine-Learning is not going anywhere

Random Forests, Logistic Regression, K-Means and friends are not going anywhere.

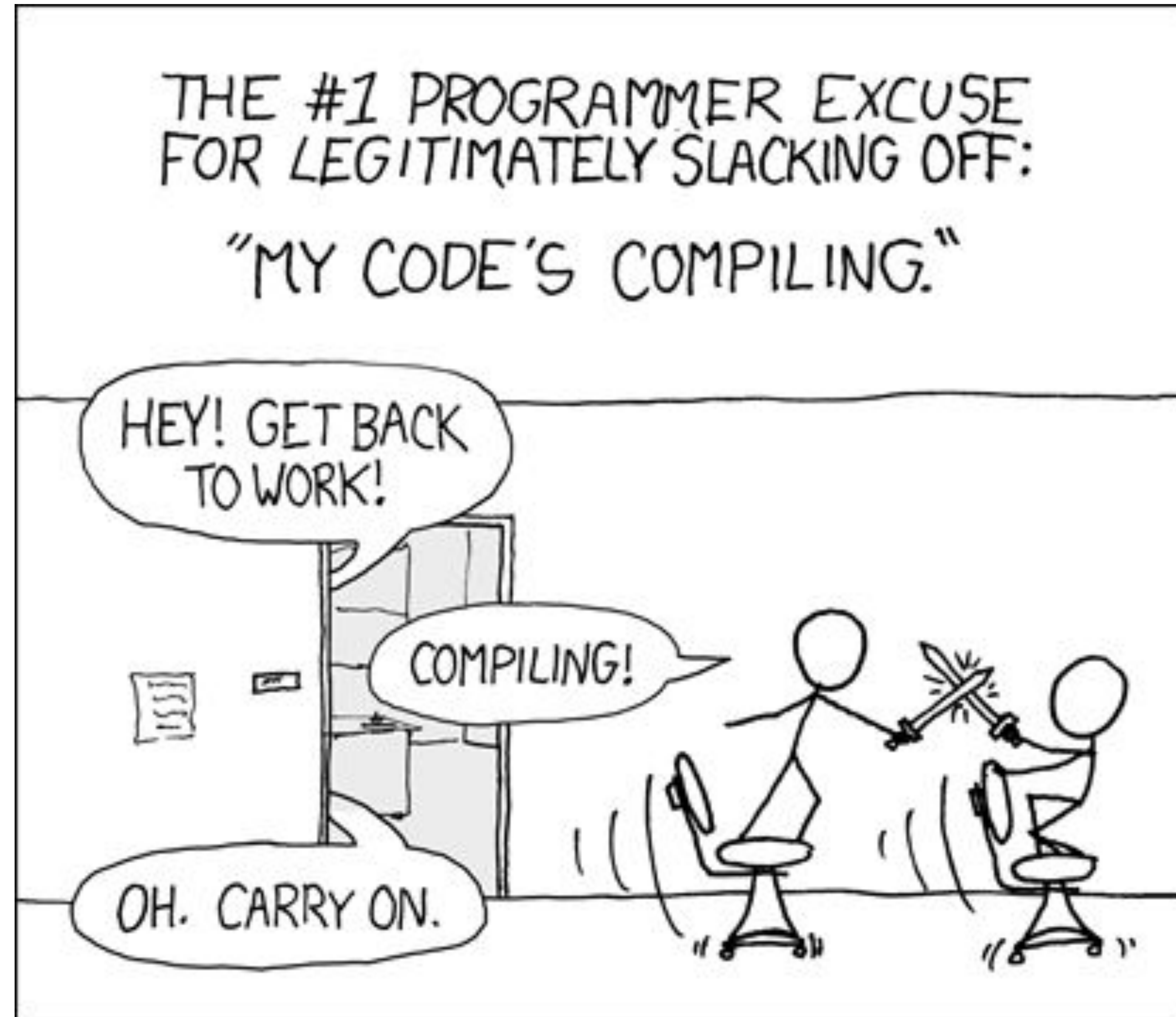
scikit-learn.org viewed by 3.5M unique people last quarter.

Topic	Views
Linear models	184 000
Clustering	112 000
Model Evaluation	106 000

What is Your Coffee Break Threshold?

Show of hands: How long can a task take before you switch context?

- 10 seconds
- 1 minute
- 10 minutes
- 100 minutes



GPU Acceleration Makes New Things Practical

New possibilities

Things become fun

Improved compatibility



RAPIDS cuML

cuML is RAPIDS' GPU-accelerated ML library that mirrors scikit-learn APIs, letting you train and deploy models on NVIDIA GPUs with minimal code changes.

Two ways you can use it:

cuml.accel - accelerate existing scikit-learn / HDBSCAN / UMAP code with no changes!

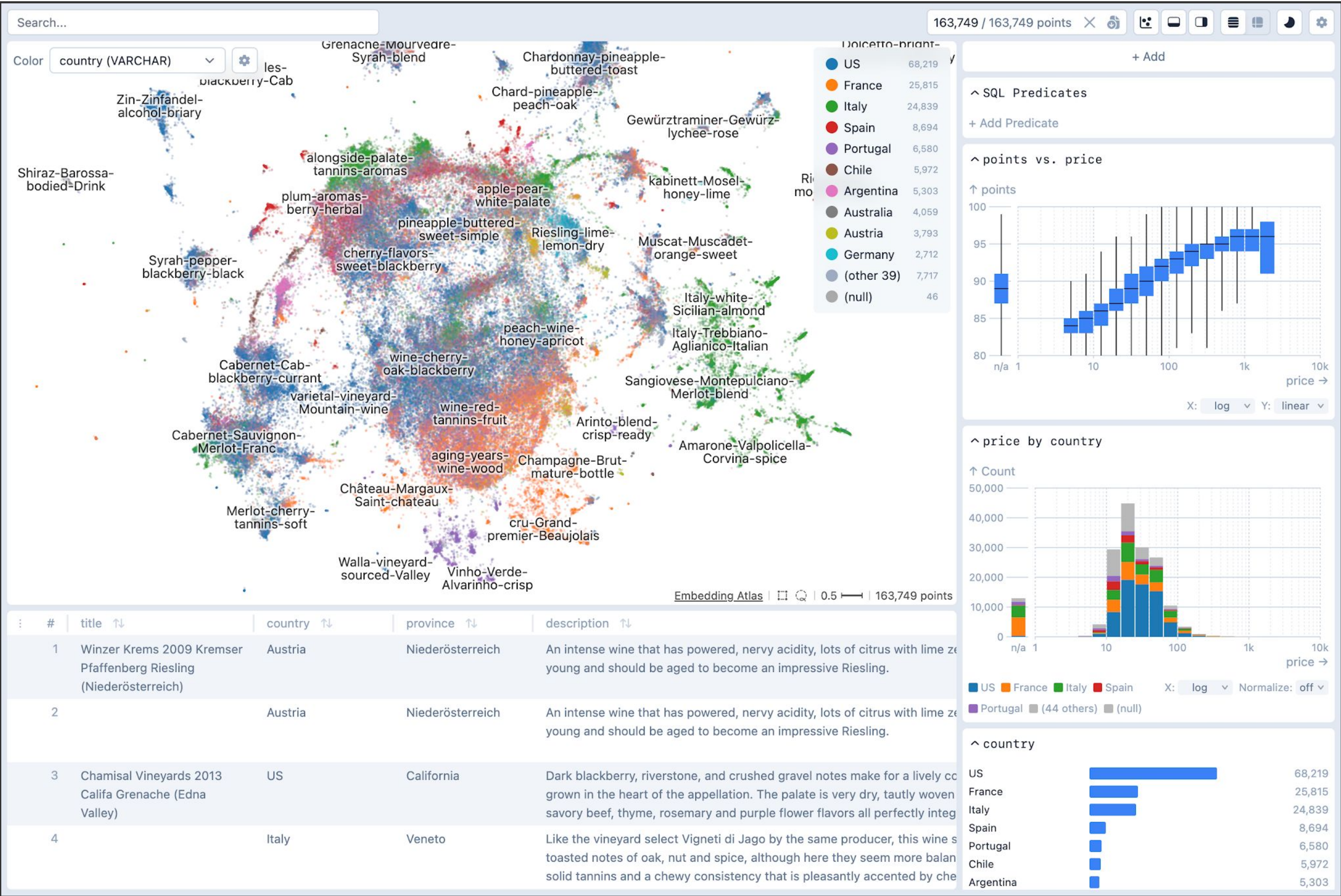
Use cuml directly - ``import cuml`` great for building new GPU accelerated code



Apple's Embedding-Atlas

Embedding Atlas is Apple's open-source interactive tool that lets you visually explore large collections of embeddings.

Uses kNN and UMAP for projections to 2D.



How to GPU accelerate Embedding Atlas

```
$ conda install -c rapidsai -c conda-forge cuml embedding-atlas
```

```
$ CUML_ACCEL_ENABLED=1 embedding-atlas ...
```

cuml.accel intercepts calls to kNN and UMAP, transparently accelerates them.

The application knows nothing about GPUs, it just got faster!

Timing results: UMAP `fit\_transform` goes from 180s -> 17s.

Using cuml directly

- Explicit is better than implicit
- Exposes hyper-parameters and algorithms that don't exist in scikit-learn, get you more performance from your GPU
- This is the opposite of the previous demo, **new code** written by you, **accelerate from day one**

```
In [1]: import this
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Complex is better than complicated.
```

```
Flat is better than nested.
```

```
Sparse is better than dense.
```

```
Readability counts.
```

```
Special cases aren't special enough to break the rules.
```

```
Although practicality beats purity.
```

```
Errors should never pass silently.
```

```
Unless explicitly silenced.
```

```
In the face of ambiguity, refuse the temptation to guess.
```

```
There should be one-- and preferably only one --obvious way to do it.
```

```
Although that way may not be obvious at first unless you're Dutch.
```

```
Now is better than never.
```

```
Although never is often better than *right* now.
```

```
If the implementation is hard to explain, it's a bad idea.
```

```
If the implementation is easy to explain, it may be a good idea.
```

```
Namespaces are one honking great idea -- let's do more of those!
```


Timseries forecasting

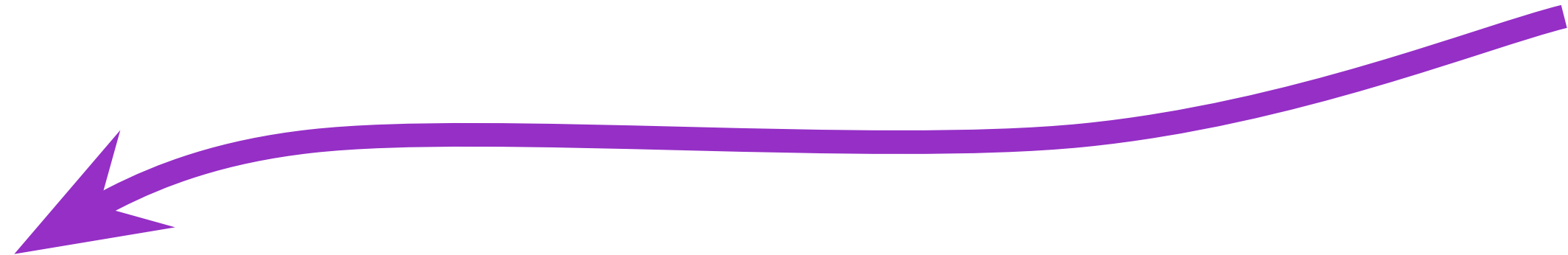
```
from sklearn.ensemble import RandomForestRegressor
from skforecast.direct import ForecasterDirect

forecaster = ForecasterDirect(
    regressor=RandomForestRegressor(
        n_estimators=100,
        max_depth=10,
        n_jobs=-1,
    ),
    steps=24,
    lags=48,
    n_jobs=1,
)

forecaster.fit(y=df['y'])
```

This takes a few minutes to run on Google Colab

Timseries forecasting with cuML



```
from cuml.ensemble import RandomForestRegressor
from skforecast.direct import ForecasterDirect

forecaster = ForecasterDirect(
    regressor=RandomForestRegressor(
        n_estimators=100,
        max_depth=10,
        n_jobs=-1,
    ),
    steps=24,
    lags=48,
    n_jobs=1,
)

forecaster.fit(y=df['y'])
```

This takes 20 seconds to run on Google Colab!

What about scikit-learn?



GPU Support in scikit-learn

- GPU support is coming!
- Current focus on using array API
 - common interface to Numpy, CuPy, PyTorch, etc
- Pass a torch CUDA tensor -> all computation happens on the GPU
- With full compatibility and usability you expect from scikit-learn

```
import os
# Enable support in Scipy
os.environ["SCIPY_ARRAY_API"] = "1" # <--

import sklearn, torch
from sklearn.linear_model import LogisticRegression

# Enable support in scikit-learn
sklearn.set_config(array_api_dispatch=True) # <--
# Use a CUDA device
X = torch.asarray(X_np, device="cuda") # <--
y = torch.asarray(y_np, device="cuda")

# Runs on the GPU
LogisticRegression().fit(X, y)
```


Estimator Tournament

```
X_np, y_np = fetch_covtype(return_X_y=True)
X_torch = torch.asarray(X_np, device="cuda", dtype=torch.float32)
y_torch = torch.asarray(y_np, device="cuda", dtype=torch.float32)
```

Where the data is determines where the computation happens.

Estimator Tournament

```
MODELS = {  
    "Logistic Regression": make_pipeline(  
        StandardScaler(),  
        LogisticRegression(solver="lbfgs", max_iter=2000),  
    ),  
    "PCA + Logistic Regression": make_pipeline(  
        StandardScaler(),  
        PCA(svd_solver="full", n_components=30),  
        LogisticRegression(solver="lbfgs", max_iter=2000),  
    ),  
    "Ridge Classifier": make_pipeline(  
        StandardScaler(),  
        RidgeClassifier(solver="svd"),  
    ),  
    "Linear Discriminant Analysis": make_pipeline(  
        StandardScaler(),  
        LinearDiscriminantAnalysis(solver="svd"),  
    ),  
    "Nyström (RBF) + Logistic Regression": make_pipeline(  
        StandardScaler(),  
        Nyström(kernel="rbf", n_components=300, random_state=42),  
        LogisticRegression(solver="lbfgs", max_iter=2000),  
    ),  
}
```

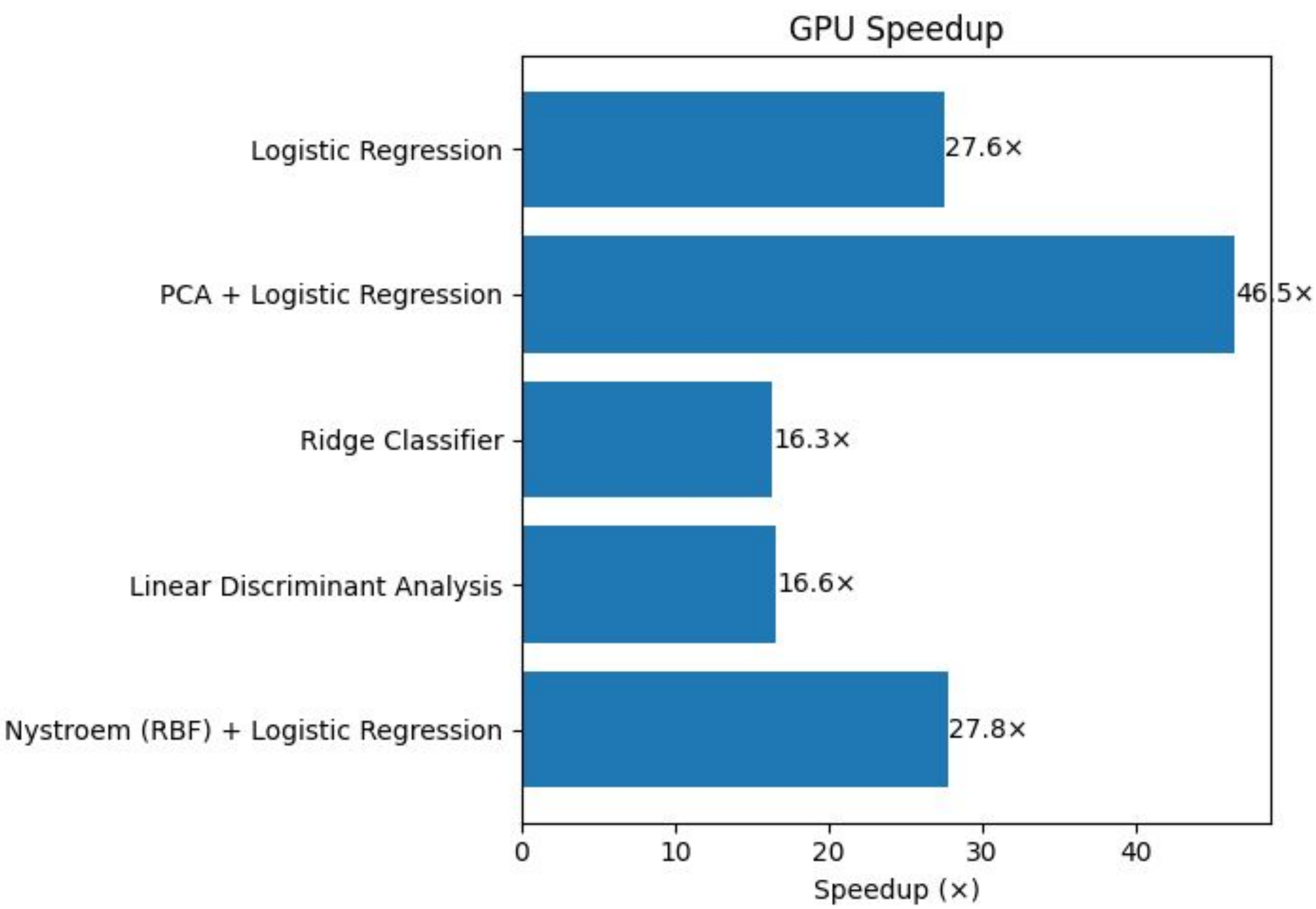
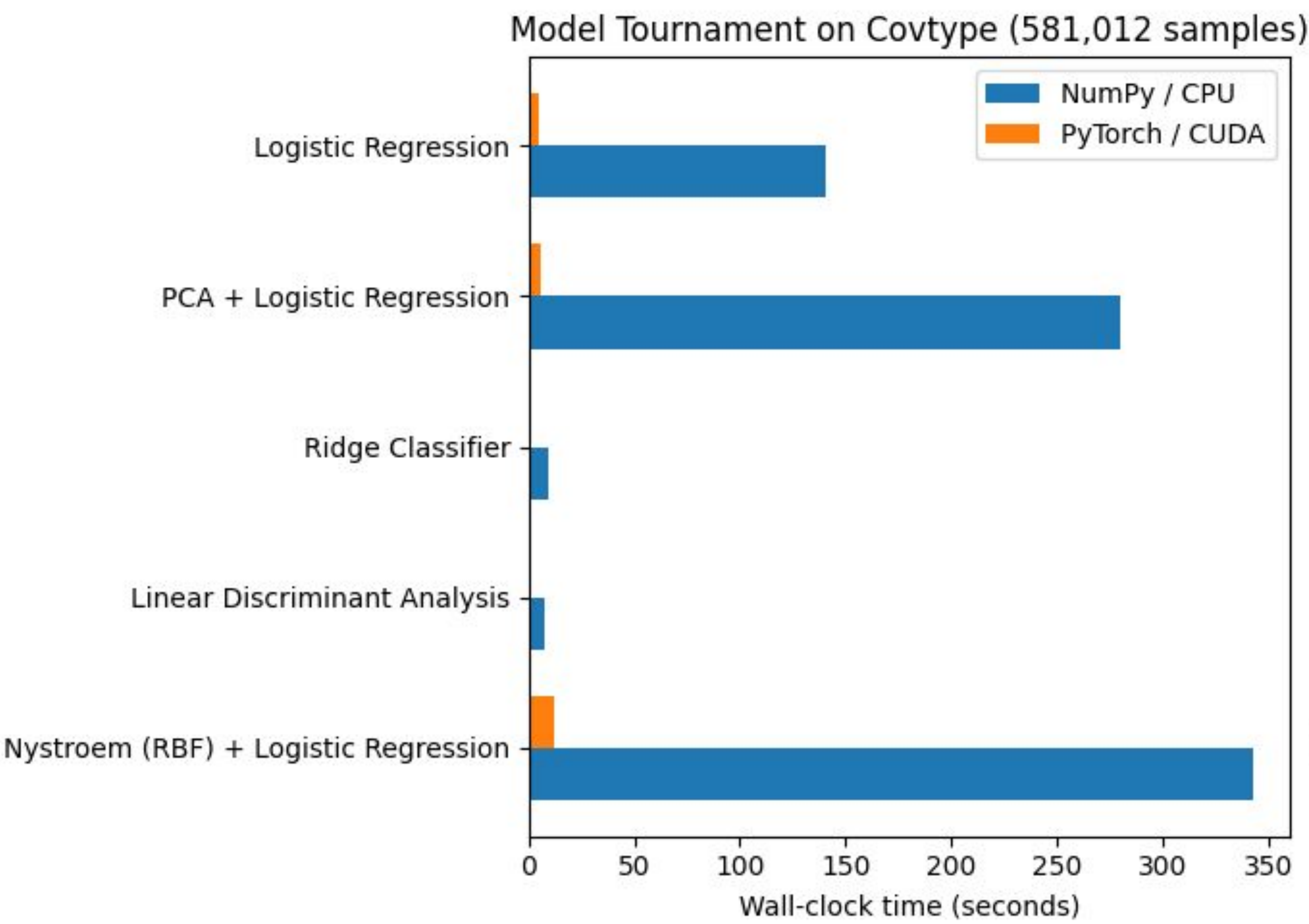

Estimator Tournament

```
# Turn on GPU support in scikit-learn
sklearn.set_config(array_api_dispatch=True)

gpu_results, gpu_total = run_tournament(X_torch, y_torch, MODELS, label="GPU")
```

By using `X\_torch` all computations happen on the GPU

Estimator Tournament



The GPU based tournament is over before you can get a coffee

What array API support gives you

- Stay in the scikit-learn ecosystem, get benefits of acceleration and reduced data movement.
- Growing list of estimators, metrics and tools.
- This is ongoing work! Send your feedback. File an issue



Compare and Contrast

cuML:

- broadest algorithm coverage on the GPU
- serves those who want maximum(!) performance
- cuml.accel is easy to try
- Apache 2 license

scikit-learn:

- familiar library
- serving a broad user base
- BSD3 license



How to try it tonight

```
from cuml.ensemble import RandomForestClassifier

clf = RandomForestClassifier(
    n_estimators=300,
    max_depth=12,
    random_state=42
)

clf.fit(X_train, y_train)
preds = clf.predict(X_test)
```

```
import os
# Enable support in Scipy
os.environ["SCIPY_ARRAY_API"] = "1" # <--

import sklearn, torch
from sklearn.linear_model import LogisticRegression

# Enable support in scikit-learn
sklearn.set_config(array_api_dispatch=True) # <--
# Use a CUDA device
X = torch.asarray(X_np, device="cuda") # <--
y = torch.asarray(y_np, device="cuda")

# Runs on the GPU
LogisticRegression().fit(X, y)
```


Classical ML isn't going anywhere

GPUs are coming to it!



